Rīgas 64. vidusskola

Mākslīgā intelekta algoritmu salīdzināšana kameras vadīšanai konferenču tiešraidēs

Zinātniskās pētniecības darbs

Inženierzinātnes un tehnoloģijas

Darba autore: Laura Žuga

Darba vadītājs: Edvards Bukovskis

Rīga 2026

Anotācija

Mākslīgā intelekta algoritmu salīdzināšana kameras vadīšanai konferenču tiešraidēs. Laura Žuga, Rīgas 64. vidusskolas 12. INZ klases skolniece. Darba vadītājs Rīgas 64. vidusskolas programmēšanas skolotājs Edvards Bukovskis.

Darba mērķis bija praktiski izveidot mākslīgā intelekta vadītu kameras sekošanas sistēmu, veikt salīdzinājumu starp dažādiem datorredzes sekošanas algoritmiem, novērtējot to ātrumu un latenci, skaitļošanas resursu patēriņu, un piemērotību konferenču tiešraidēm. Pētījuma ietvaros tika izstrādāta fiziska iekārta un programmatūra, izmantojot Python un Arduino mikrokontrolieri. Tika implementēti un testēti algoritmi CSRT, MOSSE, KCF, MediaPipe, DeepSORT, RT-DETR un YOLO + DeepSORT kombinācija, balstoties gan uz kvantitatīvajiem datiem, gan uz kvalitatīvajiem. Pētījumā secināts, ka konkrētajam uzdevumam visefektīvākais algoritms ir MediaPipe ar augstāko apstrādes ātrumu 75.18 FPS un zemāko latenci 13.6 ms.

Atslēgvārdi: datorredze, objektu sekošana, kamera, automatizācija.

Abstract

Comparison of Artificial Intelligence Algorithms for Camera Control in Conference Live Streams. Laura Žuga, Student of the 12 INZ grade, Riga Secondary School No. 64. Supervisor: Edvards Bukovskis, Programming Teacher at Riga Secondary School No. 64.

The objective of this research was to develop an artificial intelligence-controlled camera tracking system, to compare various computer vision tracking algorithms by evaluating their speed, latency, computational resource consumption, and suitability for live conference streaming. Within the framework of the research, a physical tracking device and software were developed using Python and an Arduino microcontroller. Several algorithms, including CSRT, MOSSE, KCF, MediaPipe, DeepSORT, RT-DETR, and a YOLO + DeepSORT combination, were implemented and tested using both quantitative and qualitative data. The study concluded that MediaPipe is the most effective algorithm for this specific task, demonstrating the highest processing speed of 75.18 FPS and the lowest latency of 13.6 ms.

Keywords: computer vision, object tracking, camera, automation.

Saturs

Anotācija **2**

Abstract **2**

Saturs **3**

Ievads **4**

1 Literatūras apskats **5**

1.1 Datorredzes un objektu sekošanas pamatprincipi **5**

1.2 Sekošanas algoritmi **6**

1.2.1 Klasiskie uz histogrammām balstītie algoritmi 6

1.2.2 Korelācijas filtru algoritmi 6

1.2.3 Dziļās mācīšanās sekošanas algoritmi 6

1.3 Objektu atpazīšanas algoritmi **7**

1.3.1 Tradicionālie objektu atpazīšanas algoritmi 7

1.3.2 Dziļās mācīšanās atpazīšanas algoritmi 7

1.4 Kombinētie objektu atpazīšanas un sekošanas algoritmi **7**

2 Praktiskā daļa **9**

2.1 Fiziskā iekārta **9**

2.2 Izmantotās programmas un testēšanas metodika **9**

3 Pētījuma rezultāti un to analīze **11**

3.1 Kvalitatīvais novērtējums **11**

3.2 Kvantitatīvie dati **12**

Secinājumi **14**

Izmantotie literatūras avoti **15**

Ievads

Mūsdienu digitālajā laikmetā video satura radīšana ir kļuvusi par ikdienas procesu. Strauji pieaugot pieprasījumam pēc kvalitatīva satura tiešsaistes platformās, pieaug arī nepieciešamība pēc automatizētiem risinājumiem, kas ļautu pat vienam cilvēkam radīt kvalitatīvas tiešraides. Viena no lielākajām problēmām ir dinamiska kameras vadība. Sekot kustīgam objektam vai personai, tradicionāli pieprasa atsevišķu kameras operatoru. Mākslīgā intelekta (MI) un datorredzes tehnoloģiju attīstība piedāvā inovatīvus risinājumus šai problēmai, ļaujot izveidot pilnībā vai daļēji patstāvīgas kameras sekošanas sistēmas.

Šādām sistēmām ir spēja reālajā laikā analizēt video signālu, identificēt un izsekot nepieciešamo mērķi, un pārvērst šo informāciju fiziskās komandās, lai vadītu kameras skata virzienu. Lai gan komerciāli risinājumi ir pieejami, tie bieži ir dārgi vai neelastīgi dažādām prasībām. Paša veidotās sistēmas ļauj tieši ietekmēt tās funkcijas un efektivitāti, izvēloties attiecīgās komponentes un algoritmus. Dažādi algoritmi piedāvā atšķirīgus kompromisus starp ātrumu, precizitāti un elastīgumu pret traucējumiem, piemēram, apgaismojuma maiņām vai daļēju objekta aizsegšanu. Bet, lai viss strādātu kā ieredzēts, vispirms ir būtiski izvērtēt dažādu algoritmu veiktspēju un resursu patēriņu uz konkrētās aparatūras. Tas ļauj atrast optimālo līdzsvaru starp sekošanas ātrumu un precizitāti, nodrošinot, ka izvēlētā ierīce spēj efektīvi veikt tai paredzēto uzdevumu.

Hipotēze: Optimālākais algoritms konferenču tipa pasākumiem, ņemot vērā apstrādes ātrumu, latenci, elastīgumu un nepieciešamo slodzi paredzētajam mērķim, būs CSRT algoritms.

Darba mērķis: Praktiski izveidot mākslīgā intelekta vadītu kameras sekošanas sistēmu, veikt salīdzinājumu starp dažādiem datorredzes sekošanas algoritmiem, novērtējot to ātrumu un latenci, skaitļošanas resursu patēriņu un elastīgumu.

Darba uzdevumi:

1. Apkopot un analizēt pieejamo informāciju par objektu sekošanas un atpazīšanas metodēm un robotizētu kameru vadības sistēmām.
2. Izveidot un konfigurēt fizisku sistēmu un programmu, izmantojot Python un Arduino, kas izpilda prasīto.
3. Implementēt un integrēt vairākus datorredzes sekošanas algoritmus.
4. Izstrādāt eksperimentālu metodoloģiju un testa scenārijus, lai objektīvi novērtētu katra algoritma veiktspēju pēc definētām metrikām (piemēram, latence (ms), kadru skaits sekundē (FPS), CPU noslodze (%) un noturība pret traucējumiem).
5. Veikt eksperimentus, apkopot iegūtos datus un veikt to salīdzinošo analīzi.

Izmantotās darba metodes: literatūras analīze, sistēmas projektēšana un konfigurēšana, programmēšana, eksperimenta izstrāde, datu analīze.

1 Literatūras apskats

1.1 Datorredzes un objektu sekošanas pamatprincipi

Datorredze ir mākslīgā intelekta nozare, kas pēta, kā datori var automātiski iegūt izpratni no digitāliem attēliem vai video.

Lai izprastu jebkura datorredzes algoritma darbību, primāri ir jāsaprot, kā vizuālā informācija tiek reprezentēta skaitļošanas sistēmās. Cilvēka acs uztver gaismas viļņus, ko smadzenes interpretē kā formas un krāsas, taču datoram attēls ir milzīga skaitļu matrica.

Melnbaltos attēlos katram attēlam ir viena skaitliska vērtība no 0 (melns) līdz 255 (balts). Vērtības pa vidu šim intervālam reprezentē dažādus pelēkos toņus. Šos attēlus dators redz kā 2D matricu ar dimensijām garums/platums, kur abi mērījumi ir pikseļos. (Skat. 1. att.)

Krāsainu attēlu reprezentācijai visbiežāk izmanto RGB (*Red, Green, Blue*) krāsu modeli. Šajā gadījumā attēls vairs nav 2D matrica, bet gan trīsdimensiju (3D) – 3*garums*platums. 3. dimensija ir pamata krāsu kanālu skaits. Standarta krāsu attēlam ir trīs kanāli (sarkans, zaļš, zils). Katrs kanāls darbojas kā atsevišķa matrica, kas norāda konkrētās krāsas intensitāti katrā punktā. Piemēram, spilgti sarkans pikselis datora atmiņā tiktu reprezentēts ar vektoru (255, 0, 0), kur sarkanā kanāla intensitāte ir maksimāla, bet pārējo - nulle.

255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	0	0	0	255	255	255	0	0	0	255	255	255
255	255	255	0	0	0	255	255	255	0	0	0	255	255	255
255	255	255	0	0	0	255	255	255	0	0	0	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	0	0	0	0	0	0	0	0	0	255	255	255
255	255	255	0	0	0	0	0	0	0	0	0	255	255	255
255	255	255	0	0	0	0	0	0	0	0	0	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255

1	1	1
0	0	0
-1	-1	-1

(2. att. Horizontālo malu kodols)

1	0	-1
1	0	-1
1	0	-1

(3. att. Vertikālo malu kodols)

-1	-1	-1
-1	8	-1
-1	-1	-1

(4. att. Vispārējo malu kodols)

(1. attēls. Pa kreisi redzama vizuālās informācijas reprezentācija cilvēka uztverē un datora atmiņā, Pa labi redzami konvolūcijas kodoli jeb filtri (Autora veidoti attēli))

Lai digitālajos attēlos identificētu konkrētas formas, dators izmanto konvolūcijas kodolus jeb filtrus, kas iteratīvi tiek virzīti pāri visai attēla matricai. Katrā iterācijā tiek veikta filtra šūnu vērtību reizināšana ar atbilstošajām attēla pikseļu vērtībām un iegūto reizinājumu summēšana. Rezultāts ir skaitliska vērtība, kas norāda uz atbilstību meklētajam raksturam. Jo lielāka ir rezultāta absolūtā vērtība (tālāk no 0), jo spēcīgāka ir sakritība.

Analizējot 1. attēlā izcelto apgabalu: pielietojot horizontālo malu detektoru (2. att.), tiek iegūts rezultāts 765, un, pielietojot vispārējo malu detektoru, tiek iegūts rezultāts -765. Abas absolūtās vērtības (765 un |-765|) apstiprina, ka sava pazīme ir atrasta. Savukārt vertikālo malu detektors uzrāda 0, kas precīzi norāda uz šīs pazīmes neesamību konkrētajā apgabalā.

1.2 Sekošanas algoritmi

1.2.1 Klasiskie uz histogrammām balstītie algoritmi

Šī algoritmu grupa balstās uz pieņēmumu, ka sekojamā objekta krāsu sadalījums (histogramma) ievērojami atšķiras no fona un laika gaitā mainās lēni. Algoritms iemācās iezīmētā objekta krāsu sadalījumu un nākamajā kadrā meklē vistuvāko vietu, kas atbilst definētajam objektam.

Viens no piemēriem ir CamShift *(Continuously Adaptive Mean Shift)*. Tajā attēls tiek pārveidots no RGB uz HSV (Hue, Saturation, Value) krāsu telpu. Tas ir svarīgi, jo HSV atdala krāsas toni (Hue) no spilgtuma (Value). Izmantojot tikai Hue kanālu, algoritms kļūst noturīgāks pret apgaismojuma izmaiņām (ēnām, mainīgu gaismu), jo objekta krāsas tonis mainās mazāk nekā tā RGB komponentes. No sākotnēji iezīmētā objekta tiek izveidota krāsu histogramma. Katram pikselim jaunajā kadrā tiek piešķirta varbūtība, ka tas ir daļa no iezīmētā objekta - jo pikseļa krāsa ir līdzīgāka objekta histogrammas krāsām, jo lielāka varbūtība. Vieta ar vislielāko pikseļu blīvumu, kas atbilst histogrammai ir tagad jaunā histogramma, ko izmantos nākamajam kadram. Tas dod elastīgumu, ja cilvēks pagriežas vai panāk tuvāk vai tālāk.

CamShift galvenā priekšrocība ir ātrums un zemās prasības procesoram. Taču tas balstās tikai uz krāsu, tādēļ viegli pazaudē objektu, ja fonā parādās līdzīgas krāsas objekti. Un ir jutīgs pret straujām gaismas izmaiņām, kas iziet ārpus HSV tolerances.

1.2.2 Korelācijas filtru algoritmi

Korelācijas filtru galvenais mērķis ir paātrināt objekta meklēšanu attēlā, aizstājot lēno secīgo pikseļu salīdzināšanu ar efektīvāku metodi. Tradicionāli datoram būtu jābīda meklējamā objekta paraugs pāri visam video kadram, kas prasa milzīgus resursus. Lai to apietu, korelācijas filtri izmanto Furjē transformāciju, kas attēla pikseļus pārvērš frekvencēs jeb viļņos. Šajā vidē sarežģītā salīdzināšana kļūst par vienkāršu un ļoti ātru matemātisku reizināšanu. Tiklīdz šis process ir veikts, rezultāts tiek transformēts atpakaļ pikseļos. Iegūtais rezultāts ir karte, kurā parādās viens izteikts enerģijas pīķis jeb spilgtākais punkts. Šī punkta atrašanās vieta norāda, kur attēlā atrodas meklētais objekts, tādējādi resursietilpīgu meklēšanu aizstājot ar ātru signālu apstrādi.

Papildus ātrumam šie algoritmi ir adaptīvi - tie spēj mācīties procesa gaitā. Objektam pārvietojoties un mainoties apgaismojumam vai skata leņķim, filtrs nepārtraukti atjauno savu informāciju, pievienojot klāt jauno objekta izskatu. Tas nodrošina stabilu sekošanu pat tad, ja objekts nedaudz mainās, vienlaikus saglabājot zemu datora resursu patēriņu. Bet tas arī izraisa nobīdi, laika gaitā algoritms fona objektus vai sienas var iekļaut kā daļu no oriģinālā objekta.

1.2.3 Dziļās mācīšanās sekošanas algoritmi

Dziļās mācīšanās sekošanas algoritmi fundamentāli atšķiras no klasiskajām metodēm ar to, ka tie neizmanto cilvēka definētas pazīmes, bet gan apgūst hierarhiskus objekta reprezentācijas modeļus tieši no datiem, izmantojot dziļos konvolūcijas neironu tīklus (CNN).

Dominējošā pieeja šajā jomā ir Siāmas tīklu (Siamese Networks) arhitektūra, kas vienlaicīgi apstrādā divas ievades plūsmas: mērķa šablonu un pašreizējo meklēšanas apgabalu. Abi attēli tiek izvadīti cauri identiskiem neironu tīkla slāņiem, kas ievades pikseļus transformē augstas dimensijas pazīmju iegultos, kuros iekodēta semantiskā informācija. Pēc pazīmju ekstrakcijas algoritms veic šo divu tenzoru savstarpējo korelāciju, lai ģenerētu līdzības karti. Būtiskākā priekšrocība ir tā, ka šie tīkli tiek apmācīti bezsaistes režīmā uz milzīgiem datu masīviem, iemācoties universālu līdzības funkciju, kas spēj atšķirt objektu no fona pat sarežģītos apstākļos. Sekošanas laikā tīkls veic tikai ātru izpildi, bieži vien papildinot to ar reģionu kandidātu tīkliem (RPN), lai ne tikai atrastu objekta centru, bet arī precīzi regresētu tā izmērus un formas izmaiņas.

1.3 Objektu atpazīšanas algoritmi

1.3.1 Tradicionālie objektu atpazīšanas algoritmi

Tradicionālie objektu atpazīšanas algoritmi funkcionē, balstoties uz cilvēka konstruētu pazīmju paradigmu, kur process ir sadalīts strikti determinētos un secīgos posmos, nevis vienotā apmācāmā tīklā.

Process sākas ar pazīmju inženieriju, kur algoritms no attēla ekstrahē matemātiskus deskriptorus - piemēram, HOG (orientēto gradientu histogrammas) vai SIFT (mēroga invariantiem pazīmju transformāciju). Šie deskriptori kodē attēla vizuālās primitīvas, piemēram, gradientu virzienus, malu orientāciju un tekstūras intensitāti, pārvēršot vizuālo informāciju skaitlisko vektoru kopā, kas ir noturīga pret gaismas vai mēroga izmaiņām.

Pēc tam iegūtie pazīmju vektori tiek nodoti klasiskam mašīnmācīšanās klasifikatoram, visbiežāk Atbalsta vektoru mašīnai (SVM) vai kaskādes klasifikatoram (piemēram, Viola-Jones metodē). Klasifikators, kas iepriekš apmācīts uz pozitīviem un negatīviem paraugiem, statistiski novelk lēmuma robežu (hiperplakni) daudzdimensiju telpā, lai kategorizētu, vai konkrētais pazīmju vektors atbilst meklētajam objektam vai fonam.

1.3.2 Dziļās mācīšanās atpazīšanas algoritmi

Dziļās mācīšanās (Deep Learning) objektu atpazīšanas algoritmi fundamentāli atšķiras ar to, ka tie veic automatizētu pazīmju apgūšanu. Tā vietā, lai cilvēks definētu, kā izskatās "stūris" vai "aplis", algoritms pats iemācās hierarhiskas vizuālās struktūras, izmantojot Konvolūcijas neironu tīklus (CNN).

Šo algoritmu darbību var iedalīt trīs galvenajos posmos:

- Pazīmju ekstrakcija - attēls tiek ievadīts dziļā neironu tīklā, sākuma slāņi detektē primitīvas ģeometriskas formas (līnijas, gradientus, krāsu pārejas) un dziļākie slāņi apvieno šīs primitīvas, veidojot semantiski sarežģītas reprezentācijas (acis, riteņus, tekstūras).
- Algoritmu arhitektūra - Šajā posmā algoritmi sadalās divās dominējošās saimēs atkarībā no tā, kā tie apstrādā iegūtās pazīmes: Vienpakāpes detektori (piemēram, YOLO), kas attēlu pārklāj ar virtuālu režģi un katrā šūnā vienlaicīgi prognozē gan objekta klasi, gan tā atrašanās vietu (ierobežojošo rāmi). Divpakāpju detektori, kas vispirms ģenerē reģionu priekšlikumus - vietas, kur varētu būt objekts. Pēc tam tikai šie specifiskie apgabali tiek padziļināti analizēti un klasificēti. Tas nodrošina augstāku precizitāti, bet prasa vairāk skaitļošanas resursu.
- Daudzuzdevumu zuduma funkcija - apmācības laikā tīkls vienlaicīgi risina divus matemātiskus uzdevumus: klasifikāciju (nosaka varbūtību, ka atrastais objekts pieder konkrētai kategorijai (piemēram, "kaķis 98%") un ierobežojošo rāmju regresiju (pielāgo rāmja koordinātas (x, y, w, h), lai tās maksimāli precīzi sakristu ar reālo objektu, minimizējot ģeometrisko kļūdu.)

1.4 Kombinētie objektu atpazīšanas un sekošanas algoritmi

Šo sistēmu darbības pamatā ir cikliska stāvokļa novērtēšanas un korekcijas stratēģija. Process sākas ar prognozēšanas fāzi, kurā tiek izmantots kinemātisks modelis, visbiežāk Kalmana filtrs. Tas matemātiski modelē objekta kustības trajektoriju un ātrumu, balstoties uz iepriekšējiem datiem, lai izteiktu hipotēzi par to, kur objekts atradīsies nākamajā kadrā.

Paralēli šai prognozei ik pēc noteikta intervāla tiek darbināts objektu detektors, kas neatkarīgi atrod visus attēlā esošos objektus. Svarīgākais posms šajā procesā ir datu asociācija - brīdis, kad algoritram jāsaprot, kura no tikko detektētajām "kastēm" atbilst Kalmana filtra

prognozētajai trajektorijai. Šeit tiek izmantota optimizācijas metode, piemēram, Ungāru algoritms, kas analizē ģeometrisko pārklāšanos (IoU) starp prognozēto un reālo pozīciju.

Lai novērstu kļūdas situācijās, kad objekti mainās vietām, modernie algoritmi (piemēram, DeepSORT) šo ģeometrisko analīzi papildina ar dziļo pazīmju salīdzināšanu (Re-Identification vai ReID). Neironu tīkls ģenerē unikālu "pirkstu nospiedumu" jeb vektoru katram objektam. Ja divi objekti sakrustojas, algoritms spēj tos atšķirt nevis pēc atrašanās vietas, kas ir neskaidra, bet pēc to vizuālā satura līdzības.

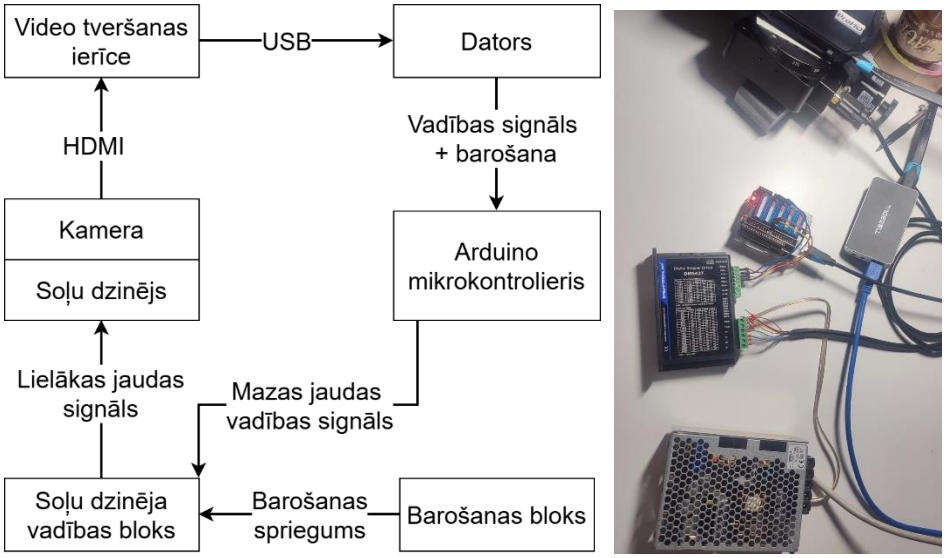
Jaunākās paaudzes pieejas (piemēram, JDE — *Joint Detection and Embedding*) šo procesu vēl vairāk optimizē, integrējot detekciju un identifikācijas vektoru ģenerēšanu vienā kopīgā neironu tīklā. Tā vietā, lai darbinātu divus atsevišķus modeļus, tīkls vienlaicīgi izvada gan objekta koordinātas, gan tā vizuālo ID vektoru, padarot sistēmu spējīgu darboties reāllaikā pat ļoti blīvā un dinamiskā vidē.

2 Praktiskā daļa

2.1 Fiziskā iekārta

Izveidotā iekārta sastāv no 7 komponentēm (skat. 5. att.)**:**

- Kamera – 720p un 50 FPS.
- Video tveršanas ierīce Magewell USB Capture HDMI Plus – nepieciešama, lai pārveidotu HDMI signālu no kameras datoram saprotamā video signālā (USB).
- Stacionārais dators: Procesors – Intel Xeon E-2136 CPU 3,3GHz, 16GB RAM, Grafiskais procesors – NVIDIA Quadro P4000.
- Arduino UNO R3 mikrokontrolieris ar komutācijas plati – darbojas kā signālu tulks. Saņem seriālos datus (Serial data) no datora par nepieciešamo rotācijas ātrumu un virzienu, un pārveido tos precīzos elektriskos impulsos dzinēja draiverim.
- Soļu dzinēja vadības bloks StepperOnline DM542T – tā kā Arduino mikrokontrolieris pats par sevi nespēj darbināt dzinēju, jo tā jauda ir pārāk maza, ir nepieciešams kontrolieris, kas pārveido mazas jaudas impulsus lielos jaudas impulsos.
- Barošanas bloks TDK Lambda RWS150B 28V 5,4A vadības blokam – nepieciešams, lai darbinātu soļu dzinēja vadības bloku
- Soļu dzinējs HT03RA100 - Saņemtos strāvas impulsus pārvērš precīzās rotācijas . Katrs impulss pagriež dzinēja asi par konkrētu leņķi (soli).



(5. att. Pa kreisi – autora veidota diagramma ar visām komponentēm, pa labi – fiziskā iekārta)

2.2 Izmantotās programmas un testēšanas metodika

Lai realizētu datorredzes un motora vadības mijiedarbību, tika izveidota vienota sistēma, kas apvieno augsta līmeņa programmēšanu (Python) ar zema līmeņa aparatūras vadību (C++ Arduino vidē).

Kā galvenā izstrādes vide (*IDE*) tika izmantots Visual Studio Code. Programma tika rakstīta Python programmēšanas valodā, izmantojot *OpenCV* bibliotēku attēlu apstrādei un *PySerial* bibliotēku datu pārraidei. Python skripts veic sekojošās funkcijas: saņem video plūsmu no kameras, apstrādā katru kadru, izmantojot izvēlēto izsekošanas algoritmu, aprēķina objekta

centru un tā nobīdi no kadra centra (kļūdu) un, ja nobīde pārsniedz noteiktu slieksni (*deadzone*), nosūta ātruma un virziena komandu uz mikrokontrolieri.

Otrā būtiskā vide ir Arduino IDE, kurā tika programmēts mikrokontrolieris. Tā uzdevums ir darboties kā tulkam starp datoru un motora draiveri, jo soļu dzinējiem nepieciešama specifiska vadība. Arduino saņemto digitālo signālu, piemēram, "griezties pa labi ar ātrumu 1000", pārveido precīzos elektriskos impulsos (*STEP/DIR* signāli). Šie impulsi tiek nosūtīti motora vadības blokam, kas attiecīgi komutē strāvu dzinēja tinumos, nodrošinot precīzu rotāciju.

Dziļās macīšanās algoritmiem vēl bija nepieciešams lejupielādēt iepriekš apmācītus svarus (*weights*), kas ļauj tiem atpazīt cilvēka aprises (klasi "Person"), nevis tikai sekot pikseļu kopai.

Pētāmie parametri jeb kvantitatīvie dati - lai novērtētu sistēmas veiktspēju, katra testa laikā tika fiksēti šādi rādītāji:

- Apstrādes ātrums - FPS jeb kadru skaits sekundē, ko algoritms spēj apstrādāt. Python kodā tika integrēts skaitītājs, kas fiksē vidējo FPS testa laikā.
- Sistēmas aizture jeb latence - laika brīdis no brīža, kad kamera uztver kadru, līdz brīdim, kad tiek nosūtīta komanda motoram.
- Resursu patēriņš jeb CPU noslodze (%) - mērīta ar Python bibliotēku *psutil*, lai noteiktu vidējo algoritma noslogojumu uz centrālo procesoru.

Pētāmie gadījumi jeb kvalitatīvais novērtējums - papildus skaitliskajiem datiem, katrs algoritms tika pārbaudīts 7 dažādos reālās dzīves scenārijos, lai novērtētu tā noturību pret traucējumiem. Tika veikti šādi testi:

1. Standarta sekošana - pārbauda algoritma pamatdarbību ideālos apstākļos.
2. Ātras kustības un dīdīšanās - pārbauda, vai algoritms neatpaliek pie straujām un netipiskām kustībām.
3. Objekta rotācija jeb mugura pret kameru - pārbauda, vai algoritms spēj turpināt sekošanu, ja pazūd galvenās iezīmes, piemēram, seja, un paliek redzams tikai ķermenis un pakausis.
4. Darbības skatuves malās - pārbauda algoritma darbību, veicot sāniskas darbības..
5. Atjaunošanās spēja (*Re-identification*) - pārbauda, vai algoritms automātiski atpazīst objektu bez lietotāja iejaukšanās, ja cilvēks vertikāli pazūd no kadra un parādās tajā no jauna.
6. Līdzīgi objekti fonā - pārbauda, vai algoritms neapjūk un nepārslēdzas uz citiem cilvēkiem vai kustīgiem objektiem fonā.
7. Aizsegšana (*Occlusion*) - pārbauda algoritma noturību brīdī, kad izsekojamajam objektam priekšā noiet cits cilvēks, vai rāmis paliek uz sākotnējā mērķa.

Pirms algoritmu testēšanas reāllaika apstākļos tika veikta sākotnējā atlase, izmantojot iefilmētu video materiālu, lai novērtētu populārāko algoritmu vispārējo piemērotību konkrētajam uzdevumam. Balstoties uz šiem rezultātiem, tālākai izpētei tika izvirzīti šādi algoritmi: CSRT, MOSSE, KCF, MediaPipe, DeepSORT, RT-DETR un YOLO + DeepSORT kombinācija. Trīs no minētajiem algoritmiem tika testēti gan ar centrālo procesoru (CPU), gan ar grafisko procesoru (GPU), lai noskaidrotu, vai GPU optimizētos modeļus ir iespējams efektīvi izmantot arī tikai ar CPU resursiem.

3 Pētījuma rezultāti un to analīze

3.1 Kvalitatīvais novērtējums

Algoritmi galvenokārt tika vērtēti pēc kvalitatīvajiem datiem, jo konferencēs tomēr ir svarīgi, lai algoritms spētu izpildīt reālistiskas situācijas. Pirmie 3 algoritmi ir korelācijas filtru algoritmi, bet pārējie ir dziļās mācīšanās algoritmi. Katrs no 7 pētāmajiem gadījumiem tika novērtēts ar krāsu, kontekstā kā tam veicās izpildīt konkrēto uzdevumu. Tumši zaļš - algoritms pilnībā izpildīja uzdevumu, gaiši zaļš - bija kāds trūkums, gaiši sarkans - bija daudzi trūkumi, bet kaut ko izpildīja, tumši sarkans - var teikt pilnībā neizpildīja. (Skat. 1. tabulu)

(1. tabula - Algoritmu kvalitatīvā novērtējuma kopsavilkums (autora veidota tabula))

Algoritms	CSRT	MOSSE	KCF	MediaPipe	DeepSORT GPU	DeepSORT CPU	RT-DETR CPU	RT-DETR GPU	YOLO + DeepSORT CPU	YOLO + DeepSORT GPU
Standarts										
Dīdīšanās										
Rotācija										
Sānisks										
Pazušana										
Cilvēks fonā										
Cilvēks priekšā										

Vienīgais algoritms, kas uzrādīja ļoti zemu veiktspēju bija dziļās mācīšanās algoritms RT-DETR, kad to mēģināja palaist bez GPU. Tas darbojās tik lēni, ka tas pat nespēja veikt sekošanu objektam, kas pārvietojas ar nesteidzīgu ātrumu. Visi pārējie algoritmi veiksmīgi tika galā ar šo pamata uzdevumu, iegūstot labāko vērtējumu.

Haotisko kustību tests atklāja ka daži tradicionālie algoritmi (CSRT, KCF) zaudēja objektu vai prasīja manuālu reģiona atlasi. MOSSE, kaut arī ātrs, nespēja pareizi interpretēt virtuves telpā notiekošās deformācijas. Savukārt DeepSORT varianti, pateicoties cilvēka klases atpazīšanai, demonstrēja labāku noturību.

Kadri, kur seja nebija redzama sagādāja grūtības lielākajai daļai algoritmu, kas balstīti uz pikseļu korelāciju vai objekta izskatu. CSRT pilnībā pazaudēja sekojamību, MOSSE uzdevumu izpildīja ar nelielu aizturi., bet KCF uzrādīja tikai nedaudz labākus rezultātus nekā CSRT. Dziļās mācīšanās algoritmi, kas analizē visu ķermeni, nevis tikai seju, tika galā daudz labāk.

Sāniskā darbība - šis tests pārbaudīja algoritmu spēju turpināt sekošanu, ja tam jāseko objekta sāna profils darbībā. YOLO + DeepSORT varianti demonstrēja stabilitāti, saglabājot objekta identitāti, kamēr tradicionālie algoritmi bieži paplašināja sekošanas lodziņu vai zaudēja precizitāti.

Identifikācijas atjaunošana izrādījās vājā vieta lielākajai daļai algoritmu. Tradicionālie algoritmi nespēja pilnībā atjaunot sekošanas rāmi bez manuālas iejaukšanās, bet daži spēja noķert pietiekami svarīgus elementus, piemēram, plecus un kaklu, lai veiksmīgi turpinātu darbu. Tikai MediaPipe, kas bija balstīts uz sejas atpazīšanu, bija vienīgais, kas kvalitatīvi spēja atgūt seju. Pārējie detektēšanas algoritmi nespēja atcerēties veco identifikāciju, veidojot jaunu.

Līdzīgu objektu klātbūtni visi algoritmi spēja atšķirt. Taču, kad tas pats cilvēks paiet garām gar priekšu, tikai viena algoritma versija bija spējīga noturēt oriģinālo seju – YOLO + DeepSORT CPU. Lai gan GPU versija bija jaudīgāka, tā nespēja tik labi noturēt objektu, savukārt CPU versijas zemākais kadru skaits (FPS) paradoksālā kārtā palīdzēja algoritmam nepārslēgties uz citu cilvēku.

3.2 Kvantitatīvie dati

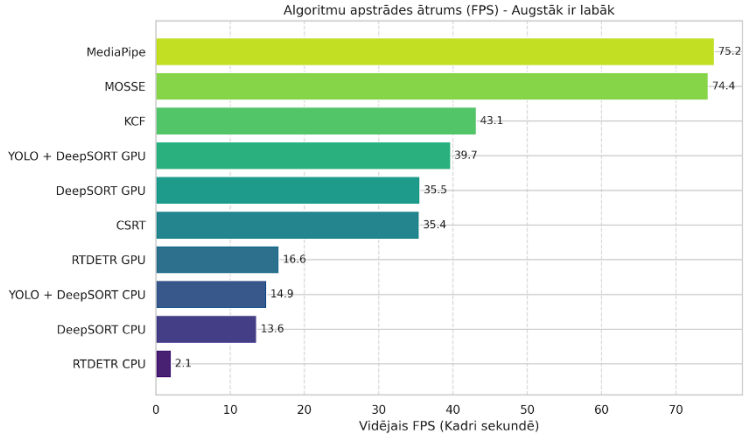
Katram MI algoritmam eksperimenta laikā tika piefiksēti apstrādes ātrums, aizture un resursu patēriņš, izmantojot vienu programmu.

Vislabāko veiktspēju uzrādīja MediaPipe ar apstrādes ātrumu 75.18 FPS un 13.60 ms latenci. Bet MOSSE bija ļoti tuvu ar apstrādes ātrumu 74.37 FPS un 13.70 ms aizturi. To zemā latence nodrošina gandrīz tūlītēju reakciju, kas ir ļoti svarīga precīzai motora vadībai. (Skat. 6., 7. att.)

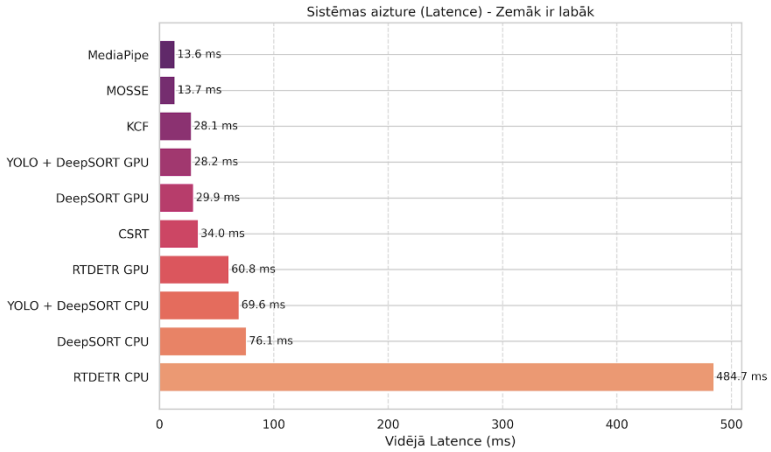
Vērts atzīmēt, ka izmantotās kameras maksimālais kadru skaits ir 50 FPS. Tā kā MediaPipe un MOSSE algoritmu apstrādes ātrums ievērojami pārsniedz šo slieksni, sistēmai veidojas veiktspējas rezerve. Tas nozīmē, ka pat ja datora noslodze palielinās citu procesu dēļ, sekošanas gludenums netiks ietekmēts, jo algoritms spēj apstrādāt kadrus ātrāk, nekā tie tiek saņemti.

GPU ietekme - dziļās mācīšanās (*Deep Learning*) algoritmiem būtiska bija grafiskā procesora izmantošana. Piemēram, YOLO + DeepSORT kombinācija uz GPU sasniedza 39.67 FPS ar pieņemamu 28.21 ms latenci, kamēr tas pats algoritms uz CPU darbojās tikai ar 14.90 FPS un augstu 69.62 ms latenci, kas radīja pamanāmu aizturi kameras kustībā. (Skat. 6., 7. att.)

Turpretī, lai gan RT-DETR CPU variantā spēja detektēt objektu, 484 ms lielā aizture padara to nederīgu reāllaika vadībai. Brīdī, kad kamera pagriežas uz cilvēka pusi, cilvēks jau ir pārvietojies tālāk. Tas rada nepatīkamu, saraustītu video efektu, kas nav pieļaujams profesionālās tiešraidēs. Savukārt MediaPipe 13.6 ms aizture ir zem cilvēka vizuālās uztveres sliekšņa, nodrošinot dabisku kameras kustību.



(6. att. - Vidējais apstrādes ātrums katram algoritmam (autora veidota diagramma))

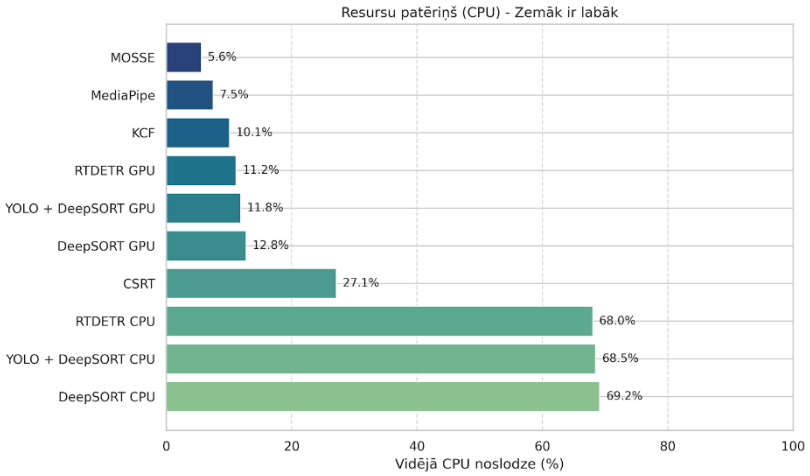


(7. att. Vidējā latence katram algoritmam (autora veidota diagramma))

Visefektīvākie resursu patēriņa ziņā bija MOSSE 5.60% un MediaPipe 7.47%, kas minimāli noslogoja centrālo procesoru, atstājot resursus citām sistēmas darbībām, piemēram, pašai straumēšanai, kad sistēma tiek finalizēta. (skat. 8. att.)

GPU izmantošana DeepSORT GPU, RTDETR GPU, YOLO+DeepSORT GPU modeļu gadījumos ļāva saglabāt zemu CPU noslodzi 11% - 12%, jo smagie matemātiskie aprēķini tika veikti videokartē. Turpretī CPU bāzētie tie paši dziļās mācīšanās algoritmi noslogoja procesoru līdz pat 69%, kas ilgtermiņā var izraisīt sistēmas pārkaršanu vai darbības traucējumus. (skat. 8. att.)

CPU noslodzes rādītāji ir būtiski, it īpaši ņemot vērā iekārtas pielietojumu. Konferenču nodrošināšanai dators ne tikai veic objektu sekošanu, bet paralēli veic video kodēšanu un straumēšanu, piemēram, uz YouTube, Facebook vai Zoom. Ja sekošanas algoritms patērē līdz pat 76% skaitļošanas resursu, straumēšanas programmatūrai var nepietikt jaudas, samazinot tiešraides kvalitāti. Tāpēc MOSSE un MediaPipe zemais patēriņš ir spēcīga priekšrocība viena datora risinājumiem.



(8. att. Vidējais CPU patēriņš katram algoritmam (autora veidota diagramma))

Secinājumi

1. Hipotēze netika apstiprināta. Pētījuma rezultāti liecina, ka visefektīvākais algoritms konkrētajam mērķim ir MediaPipe, nevis sākotnēji prognozētais CSRT, jo tas uzrādīja pārliecinoši labākos rādītājus veiktspējas mērījumos un demonstrēja visaugstāko stabilitāti visvairāk pētāmajos gadījumos.
2. Veicot sākotnējo algoritmu atlasi ar video materiāliem, GOTURN un VitTrack līdzīgiem dziļās mācīšanās algoritmiem tika novērota tendence kļūdaini vispārināt meklēšanas apgabalu. Situācijās, kad modelis apjūk, tas mēdz nepamatoti paplašināt sekošanas rāmi, identificējot visu video kadru kā izsekojamo objektu.
3. Tika secināts, ka algoritmi, kas objektu sekošanai izmanto tikai krāsu histogrammas, piemēram, CamShift, nav piemēroti iekštelpām ar siltu toņu apdari. Tā kā cilvēka ādas tonis ir ļoti līdzīgs bēšām sienām vai koka durvīm, algoritms nespēj diferencēt objektu no fona. Tas izraisa iezīmētā rāmja nekontrolējamu paplašināšanos, iekļaujot rāmī apkārtējās vides elementus.
4. Jaudīgo dziļās mācīšanās algoritmu (YOLO, DeepSORT, RTDETR) efektīvai izmantošanai ir kritiski nepieciešams grafiskais procesors (GPU), jo tie nav optimizēti CPU izmantošanai. Testi pierādīja, ka CPU režīmā šie algoritmi rada pārāk lielu latenci un noslodzi, kas padara motorizēto sekošanu saraustītu, neprecīzu un kavējošu.
5. Pat advancētie algoritmi saskaras ar grūtībām brīdī, kad viens cilvēks aizsedz otru. Lai izveidotu pilnībā automātisku sistēmu publiskām konferencēm, nepieciešami papildu uzlabojumi ID saglabāšanas loģikā, lai novērstu identifikācijas apmaiņas kļūdas.

Izmantotie literatūras avoti

Bertinetto, L., Valmadre, J., Henriques, J. F., Vedaldi, A., Torr, P. H. (2016). *Fully-Convolutional Siamese Networks for Object Tracking* [tiešsaiste]. ECCV 2016 [skatīts 2026. gada 20. novembrī]. Pieejams: https://www.robots.ox.ac.uk/~vedaldi/assets/pubs/bertinetto16fully.pdf

Bradski, G. R. (1998). *Computer Vision Face Tracking For Use in a Perceptual User Interface* [tiešsaiste]. Intel Technology Journal [skatīts 2026. gada 2. janvārī]. Pieejams: http://opencv.jp/opencv-1.0.0_org/docs/papers/camshift.pdf

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S. (2020). *End-to-End Object Detection with Transformers* [tiešsaiste]. ECCV 2020 [skatīts 2026. gada 3. janvārī]. Pieejams: https://www.ecva.net/papers/eccv_2020/papers_ECCV/papers/123560103.pdf

Chollet, F. (2018). *Deep Learning with Python*. Shelter Island: Manning Publications. 384 lpp.

Distill. (2024). *Distill Pub* [tiešsaiste]. [skatīts 2025. gada 18. Novembrī]. Pieejams: https://distill.pub/

Henriques, J. F., Caseiro, R., Martins, P., Batista, J. (2014). *High-Speed Tracking with Kernelized Correlation Filters* [tiešsaiste]. arXiv [skatīts 2026. gada 2. janvārī]. Pieejams: https://arxiv.org/pdf/1404.7584

Kalman, R. E. (1960). *A New Approach to Linear Filtering and Prediction Problems* [tiešsaiste]. Journal of Basic Engineering [skatīts 2026. gada 13. janvārī]. Pieejams: https://www.unitedthc.com/DSP/Kalman1960.pdf

LeCun, Y., Bengio, Y., Hinton, G. (2015). *Deep learning* [tiešsaiste]. Nature [skatīts 2026. gada 15. janvārī]. Pieejams: https://www.cs.toronto.edu/~hinton/absps/NatureDeepReview.pdf

Li, F., Johnson, J., Yeung, S. (2024). *CS231n: Deep Learning for Computer Vision* [tiešsaiste]. Stanford University [skatīts 2026. gada 2. janvārī]. Pieejams: https://cs231n.github.io/

Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection* [tiešsaiste]. CVPR 2016 [skatīts 2026. gada 2. janvārī]. Pieejams: https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Redmon_You_Only_Look_CVPR_2016_paper.pdf

Ren, S., He, K., Girshick, R., Sun, J. (2015). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks* [tiešsaiste]. NeurIPS 2015 [skatīts 2026. gada 3. janvārī]. Pieejams: https://proceedings.neurips.cc/paper_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf

Szeliski, R. (2022). *Computer Vision: Algorithms and Applications*. 2nd edition. Cham: Springer. 947 lpp.

Viola, P., Jones, M. (2001). *Rapid Object Detection using a Boosted Cascade of Simple Features* [tiešsaiste]. CVPR 2001 [skatīts 2026. gada 2. janvārī]. Pieejams: http://cs.cmu.edu/~efros/courses/LBMV07/Papers/viola-cvpr-01.pdf

Wojke, N., Bewley, A., Paulus, D. (2017). *Simple Online and Realtime Tracking with a Deep Association Metric* [tiešsaiste]. ICIP 2017 [skatīts 2026. gada 2. janvārī]. Pieejams: https://arxiv.org/pdf/1703.07402